Part A: Big-O Table

Method	Structures it touches	Big-O	One-line reason
addSong	HashMap, DoublyLinkedList	O(1) avg.	HashMap insertion and tail append take constant time.
findSong	HashMap	O(1) avg.	Finds a song by ID using a hash lookup.
play	HashMap, Stack	O(1) avg.	Lookup, stack push, and count update.
queueLater	HashMap, Deque	O(1) avg.	Lookup and rear insertion.
queueNext	HashMap, Deque	O(1) avg.	Lookup and front insertion.
next	Deque, HashMap, DoublyLinkedList, Stack	O(n + q)	Skips queued entries or searches the playlist.
back	Stack, HashMap, Deque	O(h)	May skip invalid history entries.
removeSong	HashMap, DoublyLinkedList	O(n)	Searches the playlist for the node to unlink.
showPlaylist	DoublyLinkedList, ArrayList	O(n)	Visits each playlist node once.

Here, n = playlist length, q = queue length, and h = history length. HashMap operations are average-case O(1).

Part B: Defend Two Choices

1. Doubly Linked List (Playlist)

I used a doubly linked list to maintain songs in playlist order. Adding a song at the tail takes O(1) time because my implementation maintains a tail reference. Finding a song to remove takes O(n), but updating the neighboring pointers takes O(1) after locating the node. A plain array would require shifting elements after deletion, which can take O(n). A built-in LinkedList provides similar functionality, but implementing my own helped me understand and manage the head, tail, and node references.

2. Deque (Up-Next Queue)

I used a deque because my music player supports adding songs to both the front and rear of the queue. The queueNext method inserts at the front, while queueLater inserts at the rear, and both take O(1) time in my implementation. A plain array would require shifting elements when

inserting at the front, taking O(n). Java's built-in ArrayDeque also supports efficient operations at both ends, but my custom implementation demonstrates how front and rear pointers work.